Equal Loss Is Not Equal: Emulator Placement, Receive Buffers, and a One-Line Congestion-Control Bug Behind an HTTP/3 Loss Penalty

Abhilash Singh
School of Computer Science and Engineering
Vellore Institute of Technology, Vellore, India

***Abstract*—Testbed comparisons of HTTP/3 over QUIC and HTTP/2 over TCP disagree about which protocol suffers more from packet loss. We started from a namespace-and-`netem` testbed in which nginx's HTTP/3 was up to $6.11\times$ slower than HTTP/2 under uniform random loss, and dissected that result with 3,060 serial, instrumented transfers across seven configurations. Three effects explain it. First, `netem` attached to a sender's own interface drops whole segmentation-offload buffers, so at a nominal 2 % loss kernel TCP experienced only 0.94 % loss events per data packet, while nginx's QUIC, which sends one datagram per buffer, experienced 1.97 %; disabling offloads with `ethtool` does not fix this, but moving `netem` to a separate router namespace does. Equalising loss shrinks the HTTP/3 penalty at 50 ms/2 % from $4.69\times$ to $1.55\times$. Second, client UDP receive-buffer overflow silently drops packets of unpaced QUIC senders and turns a 13–15 % HTTP/3 advantage at 0 % loss into a 17–36 % deficit. Third, the remaining penalty is specific to nginx: under equal loss quic-go and Cloudflare quiche are never significantly slower than TCP, and qlog traces show nginx sustaining 4 packets per round trip against quic-go's 11.5. We trace this to nginx's CUBIC computing the multiplicative decrease from bytes in flight after the acknowledged and lost packets have been removed; changing that single line makes nginx 1.5–$4.5\times$ faster under loss and removes every significant deficit to HTTP/2. We release the testbed, data and analysis as a reproducible container, and give a checklist for QUIC-vs-TCP emulation studies.**

***Index Terms*—QUIC, HTTP/3, network emulation, netem, congestion control, CUBIC, measurement methodology**

I. Introduction

QUIC [1] and HTTP/3 [2] were designed in part to perform better than TCP and HTTP/2 on lossy paths: streams are independent, so a lost packet does not block unrelated data, and loss recovery [3] benefits from unambiguous packet numbers. Measurement studies nevertheless disagree on how the two stacks compare under loss [4]–[7], and several have observed that the answer depends on the implementation and its configuration [4], [6], [8].

Many such comparisons, including course and lab projects, use the cheapest available testbed: two Linux network namespaces joined by a virtual Ethernet (veth) pair, with `tc netem` configured on the endpoints' interfaces to add delay and random loss. Our own testbed of this kind produced a clear result: at 2 % or more uniform loss, nginx's HTTP/3 was slower than HTTP/2 in every tested cell, by up to several times. This paper asks whether that verdict describes the protocols, the implementation, or the testbed.

The answer is all three, in a specific order. We make the following contributions:

- We show that `netem` on a sending endpoint applies the configured loss rate per kernel buffer rather than per packet. Linux TCP always hands the qdisc multi-segment GSO buffers and quic-go batches datagrams with UDP GSO, so both see roughly half the nominal loss events, while a sender without batching sees all of them (§IV). `ethtool` offload settings do not change this; a separate router namespace does.
- We show that client UDP receive-buffer overflow is a second, host-dependent source of loss that affects only unpaced QUIC senders and reverses the 0 %-loss verdict (§V).
- Under equal per-packet loss, two mature QUIC stacks (quic-go, quiche) match or beat kernel TCP in every cell, while nginx remains up to $2.72\times$ slower (§VI).
- Using qlog and a one-line intervention, we trace nginx's deficit to the way its CUBIC multiplicative decrease reads bytes in flight, and quantify the effect of changing it (§VII).
- We publish the full testbed as a pinned container, all 3,060 transfer records with per-transfer kernel counters, and the scripts that generate every number in this paper.

II. Background

netem and segmentation offload. `netem` is a queueing discipline: it decides whether to drop a packet when the kernel enqueues a socket buffer (skb) on the device [9]. With generic segmentation offload (GSO) [10], the TCP stack builds skbs that carry many MSS-sized segments and splits them only after the qdisc, or in the NIC. Linux TCP always uses GSO internally, whatever the device advertises. On the UDP side, QUIC stacks such as quic-go use UDP GSO (`UDP_SEGMENT`) to pass several datagrams to the kernel in one call. One `netem` "loss" on such an skb removes a burst of packets that the sender then detects as a single loss event. `netem`'s loss accuracy has

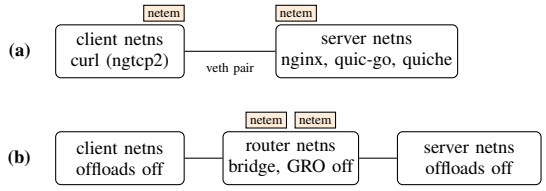


Fig. 1. Topologies. (a) `netem` on the endpoints' own veth devices (configurations A, B; the original testbed). (b) `netem` on the egress ports of a separate router namespace, so it sees wire-sized packets (F, G, D, P, H). Each `netem` adds half the RTT, the loss rate and a 1 Gbit/s rate in one direction.

been studied in isolation [11]; its interaction with sender-side batching has, to our knowledge, only been discussed informally.

Receive buffers. On fast emulated links, a user-space QUIC receiver may not drain its UDP socket fast enough; datagrams that do not fit into the socket buffer are counted in `Udp RcvbufErrors` and dropped before QUIC sees them. Zhang et al. showed that receiver-side processing limits QUIC on fast links [12]; Jaeger et al. examined buffer sizes and offloads for QUIC throughput [13], [14].

CUBIC multiplicative decrease. On a congestion event CUBIC sets $ssthresh = \beta \cdot flight_size$ with $\beta = 0.7$, where flight size is the amount of outstanding data [15], [16]. RFC 9438 permits implementations to use the congestion window instead, provided they prevent the window from growing while the sender is application-limited. RFC 9002's reference pseudocode for QUIC reduces from the congestion window [3].

III. Testbed and Method

Testbed. Fig. 1 shows the two topologies. All software runs in one privileged Docker container (Ubuntu 22.04) on Docker Desktop's Linux 6.10 VM (8 vCPUs, 8 GB) on an Apple M3 host. The client is curl 8.23.1-DEV built with ngtcp2, nghttp3 and OpenSSL 3.6.5 [17]. Four senders serve the same 1 MB object over TLS: nginx 1.31.9 over HTTP/2 (kernel TCP, CUBIC) and over HTTP/3 using its native QUIC module [18], a quic-go v0.64.1 HTTP/3 file server [19], and Cloudflare quiche 0.29.3's `quiche-server` [20]. All QUIC senders use CUBIC. Every source is pinned to an exact commit and built in a Dockerfile.

Procedure. Each transfer is a fresh curl process and connection, so completion time includes the handshake. Transfers run strictly one at a time. The client and each server are pinned to separate vCPUs, and the order of senders is rotated every iteration to cancel drift. The grid is RTT $\in \{50, 100\}$ ms $\times$ uniform loss $\in \{0, 1, 2, 5\}$ % applied in both directions, with 20 iterations per sender and cell and a 40 s cap (no transfer hit it). Before measuring, each sender must serve the object over the intended HTTP version on a clean link. For every transfer we record curl's timings, client and server CPU time, and kernel counters: `netem` sent and dropped packets in each direction, TCP `RetransSegs`, and client `Udp RcvbufErrors`. The realised loss rate is therefore measured rather than assumed.

Statistics. Because absolute times drift between runs, the primary metric is the median of per-iteration paired ratios

TABLE I
Configurations. n = transfers after exclusions.

Cfg	netem on	Offl.	rcvbuf	HTTP senders	n
A	endpoint veth	on	208 KB	h2, nginx, quic-go	479
B	endpoint veth	off	208 KB	h2, nginx, quic-go	480
F	router netns	off	208 KB	h2, nginx, quic-go	480
G	router netns	off	4 MB	h2, nginx, quic-go	480
D	router netns	off	4 MB	nginx, quic-go + qlog	30
P	router netns	off	4 MB	h2, *patched* nginx, quic-go	477
H	router netns	off	4 MB	h2, nginx, quic-go, quiche	634

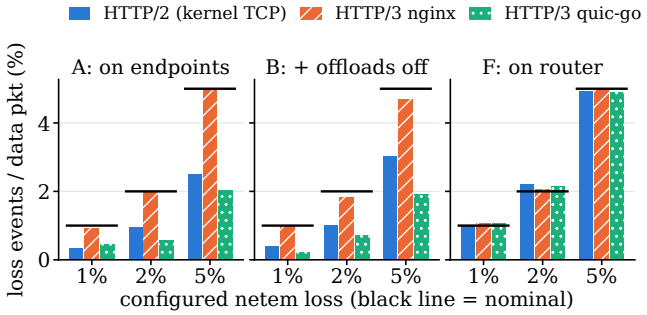


Fig. 2. Realised loss events per data packet (`netem` drops on the data path / packets offered), median over 20 transfers per bar, 50 ms RTT. On an endpoint (A, B) TCP and quic-go see about half the configured loss; on a router (F) all senders see it.

(HTTP/3 time / HTTP/2 time in the same iteration), with a 2000-sample bootstrap 95 % CI. A difference is called significant when the CI excludes 1 and a Wilcoxon signed-rank test is significant at 0.05 after Bonferroni correction over all comparisons in that configuration (16, or 24 for H).

Data hygiene. The host idled to sleep three times during unattended runs, suspending the VM. We drop any transfer whose wall-clock gap to the previous transfer exceeds its own duration by more than 60 s (10 transfers in total); later runs prevented sleep. The process-hang guard never fired, and there were 0 failed transfers.

Table I lists the configurations. A reproduces the original testbed settings. B turns endpoint offloads off with `ethtool`. F moves `netem` to the router. G additionally raises the client's default UDP receive buffer from 208 KB to 4 MB. D repeats one cell of G with qlog [21] enabled. P is G with a one-line change to nginx (§VII). H is G with all four senders in the same run.

IV. Pitfall 1: Where netem Sits

Fig. 2 shows the loss each sender actually experienced. With `netem` on the endpoint (A), a nominal 2 % produced 0.94 % loss events per data packet for TCP, 0.57 % for quic-go and 1.97 % for nginx's QUIC; at 5 % the figures are 2.51, 2.02 and 5.01 %. nginx sends one datagram per system call, so each of its packets is its own skb. TCP and quic-go hand the qdisc multi-packet buffers, and one drop decision removes the whole buffer. To the sender, a burst of consecutive losses within one round trip costs about as much as a single loss: both TCP

and QUIC reduce their window once per round trip. So in the original testbed, nginx's QUIC was in effect tested at roughly twice the loss rate of TCP.

Turning off TSO, GSO and GRO on the endpoint veths (B) barely changes this (TCP 1.01 %, quic-go 0.72 %, nginx 1.83 % at 2 %). The device features only control whether segmentation happens in the device. The TCP stack and `UDP_SEGMENT` still build multi-packet skbs above the qdisc, and software segmentation then happens after `netem` has decided. Only moving `netem` off the senders equalises loss. In F, packets are segmented before they leave the sender namespace, the router does not re-aggregate them, and all three senders see 2.20, 2.05 and 2.15 % at a nominal 2 %.

Effect on the verdict. Table II compares A and F. nginx's own completion time is unchanged (7.28 s in A vs. 7.38 s in F at 50 ms/2 %), while HTTP/2 becomes much slower (1.50 s to 4.65 s) once it receives the full loss. The HTTP/3/HTTP/2 ratio falls from 4.69 to 1.55 at 2 % and from 2.50 to 1.08 (not significant) at 5 %. Most of the reported HTTP/3 loss penalty was TCP being tested at a lower loss rate.

V. Pitfall 2: The Receiver's UDP Buffer

At 0 % configured loss, nginx's HTTP/3 transfers in A lost a median of 188 datagrams each to client receive-buffer overflow, while TCP and quic-go lost none. In F, nginx accumulated 7,673 `RcvbufErrors` over its transfers against 12 for quic-go. nginx bursts a full window without pacing into a 208 KB default buffer, while quic-go paces its sending. These drops are invisible to `netem`'s counters and act as extra, sender-specific loss.

With a 4 MB buffer (G, H, P) we observed 0 receive-buffer errors, and the 0 %-loss verdict reverses: nginx HTTP/3 becomes 13–15 % *faster* than HTTP/2, consistent with QUIC's one-round-trip handshake saving [22], where in F it was 17–36 % slower. The default buffer size differs between distributions and virtualisation layers, so this verdict can change between hosts without any change to the protocols. Median client CPU time per 1 MB transfer was 0.016–0.022 s for all senders, so with serial transfers CPU was not the bottleneck.

VI. Under Equal Loss, the Implementation Decides

With loss equalised and receive buffers fixed, Table II (columns G and H) and Fig. 3 separate the senders cleanly. In H, where all four senders share one run:

- quic-go is never significantly slower than HTTP/2 (ratios 0.54–0.94) and is significantly faster in 5 of 8 cells;
- quiche is never significantly slower (0.70–1.01) and is significantly faster in 3 of 8;
- stock nginx is significantly slower in 4 of 6 lossy cells (up to 2.62×), and faster only at 0 % loss.

Fig. 4(a) shows the full distributions at 50 ms/2 %. The two mature QUIC stacks complete faster than TCP across the whole distribution, and nginx is slower across the whole distribution. nginx's pattern in H matches G, which ran hours earlier, so it is not run-to-run drift. The HTTP/3 loss penalty seen in the original testbed is therefore a property of one QUIC sender, made larger by two testbed artefacts. It is not a property of the protocol.

Listing 1. Congestion-event handler in nginx's QUIC module (abridged)

```
blocked = (cg->in_flight >= cg->window) ? 1 : 0;
cg->in_flight -= f->plen;       /* lost packet removed */
...
/* RFC 9438, 4.6. Multiplicative Decrease */
cg->recovery_start = now;
cg->w_prior = cg->window;
cg->w_max = (cg->window < cg->w_max)
          ? cg->window * (10 + NGX_QUIC_CUBIC_BETA) / 20
          : cg->window;
/* P changes in_flight to window in the next line */
cg->ssthresh = cg->in_flight * NGX_QUIC_CUBIC_BETA / 10;
cg->window = ngx_max(cg->ssthresh, cg->mtu * 2);
```

VII. Root Cause in nginx's CUBIC

Symptom. Run D repeats 50 ms/2 % with qlog enabled at the client and on the quic-go server (nginx does not emit qlog). Neither QUIC sender shows long idle periods driven by probe timeouts. What differs is the steady sending rate. The client receives a median of 4.0 packets per 50 ms round trip from nginx and 11.5 from quic-go, whose congestion window has a time-weighted median of 13.9 packets. An AIMD sender at $p = 0.02$ is expected to sustain about $1.22/\sqrt{p} \approx 8.6$ packets per round trip [23]. nginx sends at less than half of that, so its window is held below what its own CUBIC should reach.

Cause. Listing 1 shows nginx's congestion-event handler. It runs from loss detection, which nginx performs at the end of processing an ACK frame. By then the bytes of every newly acknowledged packet have been subtracted from `in_flight`, and the handler subtracts the lost packet's bytes before computing the new threshold. When an ACK both acknowledges a run of packets and declares one lost, which is the common case under random loss, `in_flight` at that point can be well below both the window and the amount outstanding at the moment of the loss. The decrease then becomes much deeper than β, and it repeats at every congestion event.

Intervention. We rebuilt the same nginx commit with only the marked line changed to reduce from `cg->window`, which is the reduction in RFC 9002's reference pseudocode, and repeated G (configuration P). Table III shows the result. Loss-free performance is unchanged, because the change touches only the congestion-event path. Under loss, nginx becomes 1.5–4.5× faster. Its ratio to HTTP/2 drops to 0.64–1.31 (Table II, column P). It is significantly slower than HTTP/2 in 0 of 6 lossy cells, compared with 5 for stock nginx in G, and significantly faster in 2. Fig. 4(b) shows the shift at 100 ms/2 %.

Interpretation. Reducing from bytes in flight is permitted by RFC 9438 [15]; the defect is *when* nginx samples it. Our change shows that this one line explains most of nginx's deficit. We do not propose it as the upstream fix: a sender that reduces from the window must also prevent window growth while application-limited, and we have not audited nginx's growth path for that. A conservative fix would record the flight size before the ACK frame is processed and use that value in the

TABLE II

MEDIAN PAIRED COMPLETION-TIME RATIO HTTP/3 / HTTP/2 PER CELL AND CONFIGURATION (< 1: HTTP/3 FASTER). **BOLD**$^{\uparrow}$/**BOLD**$^{\downarrow}$: SIGNIFICANTLY SLOWER/FASTER (BOOTSTRAP CI EXCLUDES 1 AND BONFERRONI-CORRECTED WILCOXON $p < 0.05$). SECOND COLUMN: HTTP/2 MEDIAN TIME IN G FOR SCALE. †P USES THE PATCHED NGINX (§VII).

	HTTP/2	nginx HTTP/3 / HTTP/2					quic-go / HTTP/2				quiche
RTT, loss	G (s)	A	F	G	H	P†	A	F	G	H	H
50 ms, 0 %	0.55	**1.23**$^{\uparrow}$	**1.17**$^{\uparrow}$	**0.85**$^{\downarrow}$	**0.79**$^{\downarrow}$	**0.81**$^{\downarrow}$	**0.62**$^{\downarrow}$	**0.61**$^{\downarrow}$	**0.63**$^{\downarrow}$	**0.60**$^{\downarrow}$	**0.84**$^{\downarrow}$
50 ms, 1 %	1.62	**3.54**$^{\uparrow}$	2.10	**2.72**$^{\uparrow}$	**1.86**$^{\uparrow}$	1.31	0.65	0.88	0.70	0.61	0.97
50 ms, 2 %	4.29	**4.69**$^{\uparrow}$	**1.55**$^{\uparrow}$	**1.63**$^{\uparrow}$	**1.49**$^{\uparrow}$	0.84	0.76	**0.61**$^{\downarrow}$	0.62	**0.62**$^{\downarrow}$	0.84
50 ms, 5 %	9.58	**2.50**$^{\uparrow}$	1.08	**1.15**$^{\uparrow}$	1.22	**0.64**$^{\downarrow}$	**0.52**$^{\downarrow}$	**0.50**$^{\downarrow}$	**0.57**$^{\downarrow}$	**0.58**$^{\downarrow}$	**0.78**$^{\downarrow}$
100 ms, 0 %	0.99	**1.34**$^{\uparrow}$	**1.36**$^{\uparrow}$	**0.87**$^{\downarrow}$	**0.89**$^{\downarrow}$	**0.87**$^{\downarrow}$	**0.68**$^{\downarrow}$	**0.70**$^{\downarrow}$	**0.69**$^{\downarrow}$	**0.70**$^{\downarrow}$	0.89
100 ms, 1 %	4.44	**4.23**$^{\uparrow}$	**1.99**$^{\uparrow}$	**2.07**$^{\uparrow}$	**2.62**$^{\uparrow}$	0.66	0.85	0.76	0.77	0.94	1.01
100 ms, 2 %	7.72	**6.11**$^{\uparrow}$	**2.02**$^{\uparrow}$	**1.60**$^{\uparrow}$	**1.51**$^{\uparrow}$	**0.65**$^{\downarrow}$	0.78	**0.50**$^{\downarrow}$	0.64	0.79	0.85
100 ms, 5 %	17.04	**1.88**$^{\uparrow}$	1.10	1.23	1.09	0.87	**0.56**$^{\downarrow}$	**0.59**$^{\downarrow}$	**0.59**$^{\downarrow}$	**0.54**$^{\downarrow}$	**0.70**$^{\downarrow}$

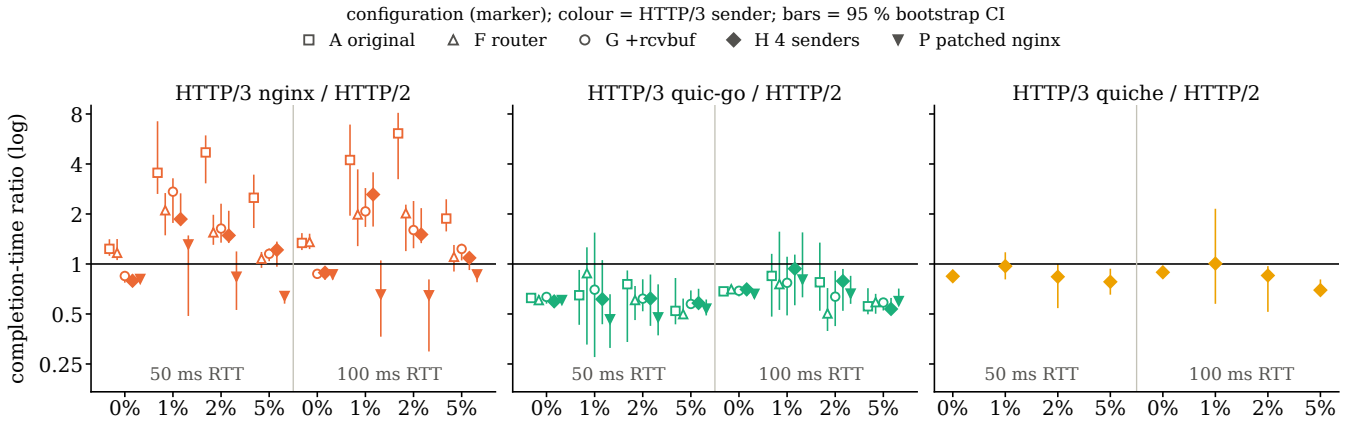


Fig. 3. Paired HTTP/3/HTTP/2 completion-time ratios with 95 % bootstrap CIs per sender (panel), cell (x) and configuration (marker). Moving `netem` to a router (A→F) collapses most of nginx's penalty. quic-go and quiche stay at or below 1 throughout. Patching nginx (P) moves it below 1 under loss.

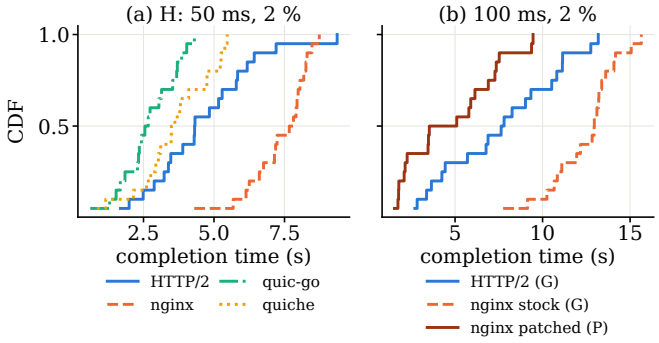


Fig. 4. Completion-time CDFs, 20 transfers per line. (a) All four senders in one run (H) at 50 ms RTT, 2 % loss. (b) Stock nginx (G), patched nginx (P) and HTTP/2 (G) at 100 ms, 2 %.

TABLE III

MEDIAN NGINX HTTP/3 COMPLETION TIME, STOCK (G) VS. ONE-LINE PATCH (P).

RTT, loss	stock (G), s	patched (P), s	speed-up
50 ms, 0 %	0.47	0.47	1.00×
50 ms, 1 %	4.58	2.05	2.24×
50 ms, 2 %	6.86	3.77	1.82×
50 ms, 5 %	11.19	6.73	1.66×
100 ms, 0 %	0.87	0.88	0.99×
100 ms, 1 %	8.14	1.81	4.49×
100 ms, 2 %	12.95	4.31	3.00×
100 ms, 5 %	20.40	13.73	1.49×

decrease. We plan to report the issue to the nginx maintainers with this data.

VIII. RELATED WORK

Early QUIC-vs-TCP comparisons found QUIC ahead on lossy and high-delay paths, with large variation across configurations and versions [4], [7], [22], [24]. Kakhki et al. [4] stressed calibrated, instrumented comparisons. Wolsing et al. [5] argued that fair comparisons need equally tuned stacks. Yu and Benson [6] showed that production QUIC performance depends heavily on implementation and congestion-control choices. Marx et al. [8] documented wide behavioural diversity across 15 QUIC implementations and introduced qlog-based root-cause analysis, which we use. Jaeger et al. and Kempf et al. [13], [14] studied buffer sizes and offloads for QUIC throughput on high-rate links, and Zhang et al. [12] identified

receiver-side limits. Endres et al. [25] found large differences between QUIC stacks under loss on satellite paths. Our work differs in focus: we quantify how emulator placement biases the *loss* each protocol experiences, show that `ethtool` does not remove the bias, and carry one implementation's residual penalty down to a single line of code with an intervention.

IX. Recommendations

For QUIC-vs-TCP comparisons on Linux emulation testbeds:

1) Do not attach `netem` to a sender's own interface. Use a separate forwarding namespace or host with GRO off, and segment on the endpoints.
2) Measure realised loss per sender from `netem`'s counters, or a capture, rather than reporting the configured rate.
3) Log `Udp RcvbufErrors` and size the client's socket buffer explicitly; report it.
4) Compare more than one QUIC implementation before attributing a result to the protocol, and record each one's congestion controller and pacing.
5) Run transfers serially, pin processes, rotate sender order, and use within-iteration paired comparisons.
6) Check host power management and wall-clock gaps on laptops and VMs.

X. Limitations

All runs used one host type, Docker Desktop's Linux VM on Apple silicon. The placement effect follows from kernel mechanisms that are not specific to it, but we have not yet repeated A vs. F on bare metal or on the original WSL2 host. We test one object size (1 MB), uniform random loss, two RTTs, an unconstrained 1 Gbit/s rate (no bottleneck queue), CUBIC everywhere, and a single client implementation, so ACK-frequency and flow-control effects on the client side are not varied. Configurations ran sequentially over about a day, so comparisons of absolute times across configurations include drift; our conclusions rest on within-run paired ratios, and on H, which compares all senders in one run. We infer nginx's sending rate from client-side qlog because nginx does not export qlog.

XI. Conclusion

A simple namespace testbed reported that HTTP/3 suffers badly under packet loss. Most of that result came from where `netem` was attached and from the client's receive buffer. The rest came from one line in one implementation's congestion controller. With equal loss, two mature QUIC stacks match or beat kernel TCP, and nginx does too once that line is changed. Testbed papers comparing transports should measure the loss they actually apply, and should not attribute a single implementation's behaviour to the protocol.

Artifact Availability

The container definitions, harness, raw per-transfer CSVs, qlog traces and the script that generates every number, table and figure in this paper are available at https://github.com/abhi-s99/quic-testbed (branch `rigor/clean-rerun`).

References

[1] J. Iyengar and M. Thomson, "QUIC: A UDP-based multiplexed and secure transport," RFC 9000, IETF, 2021.
[2] M. Bishop, "HTTP/3," RFC 9114, IETF, 2022.
[3] J. Iyengar and I. Swett, "QUIC loss detection and congestion control," RFC 9002, IETF, 2021.
[4] A. M. Kakhki, S. Jero, D. Choffnes, C. Nita-Rotaru, and A. Mislove, "Taking a long look at QUIC: An approach for rigorous evaluation of rapidly evolving transport protocols," in *Proc. ACM Internet Measurement Conference (IMC)*, 2017.
[5] K. Wolsing, J. Rüth, K. Wehrle, and O. Hohlfeld, "A performance perspective on web optimized protocol stacks: TCP+TLS+HTTP/2 vs. QUIC," in *Proc. Applied Networking Research Workshop (ANRW)*, 2019.
[6] A. Yu and T. A. Benson, "Dissecting performance of production QUIC," in *Proc. The Web Conference (WWW)*, 2021.
[7] P. Megyesi, Z. Krämer, and S. Molnár, "How quick is QUIC?" in *Proc. IEEE International Conference on Communications (ICC)*, 2016.
[8] R. Marx, J. Herbots, W. Lamotte, and P. Quax, "Same standards, different decisions: A study of QUIC and HTTP/3 implementation diversity," in *Proc. Workshop on the Evolution, Performance, and Interoperability of QUIC (EPIQ)*, 2020.
[9] "net/sched/sch_netem.c, Linux kernel 6.10," https://git.kernel.org/, 2024.
[10] "Segmentation offloads (Documentation/networking/segmentation-offloads.rst)," Linux kernel documentation, https://docs.kernel.org/networking/segmentation-offloads.html.
[11] A. Jurgelionis, J.-P. Laulajainen, M. Hirvonen, and A. I. Wang, "An empirical study of NetEm network emulation functionalities," in *Proc. International Conference on Computer Communications and Networks (ICCCN)*, 2011.
[12] X. Zhang, S. Jin, Y. He, A. Hassan, Z. M. Mao, F. Qian, and Z.-L. Zhang, "QUIC is not quick enough over fast internet," in *Proc. ACM Web Conference (WWW)*, 2024.
[13] B. Jaeger, J. Zirngibl, M. Kempf, K. Ploch, and G. Carle, "QUIC on the highway: Evaluating performance on high-rate links," in *Proc. IFIP Networking Conference*, 2023.
[14] M. Kempf *et al.*, "QUIC on the fast lane: Extending performance evaluations on high-rate links," *Computer Communications*, 2024.
[15] L. Xu, S. Ha, I. Rhee, V. Goel, and L. Eggert, "CUBIC for fast and long-distance networks," RFC 9438, IETF, 2023.
[16] M. Allman, V. Paxson, and E. Blanton, "TCP congestion control," RFC 5681, IETF, 2009.
[17] "curl 8.23.1-dev with ngtcp2 and nghttp3," https://github.com/ngtcp2/ngtcp2, 2026.
[18] "nginx, commit 797c039 (1.31.9), QUIC module src/event/quic," https://github.com/nginx/nginx, 2026.
[19] "quic-go v0.64.1," https://github.com/quic-go/quic-go, 2026.
[20] "Cloudflare quiche 0.29.3," https://github.com/cloudflare/quiche, 2026.
[21] R. Marx, L. Niccolini, M. Seemann, and L. Pardue, "qlog: Structured logging for network protocols," IETF Internet-Draft draft-ietf-quic-qlog-main-schema, work in progress.
[22] A. Langley *et al.*, "The QUIC transport protocol: Design and internet-scale deployment," in *Proc. ACM SIGCOMM*, 2017.
[23] M. Mathis, J. Semke, J. Mahdavi, and T. Ott, "The macroscopic behavior of the TCP congestion avoidance algorithm," *ACM SIGCOMM Computer Communication Review*, vol. 27, no. 3, 1997.
[24] P. Biswal and O. Gnawali, "Does QUIC make the web faster?" in *Proc. IEEE Global Communications Conference (GLOBECOM)*, 2016.
[25] S. Endres *et al.*, "Performance of QUIC implementations over geostationary satellite links," arXiv:2202.08228, 2022.